

AMATH 582: Home Work 4

Mengze Yu, Email address: mengzy3@uw.edu

This report is devoted to test the ability of FCN method to classify the different figures from the FashionMNIST dataset. What's more, the report will also show the effect of different parameters, optimizers, initial conditions and whether the dropout strategy or batch normalization is used. More specifically, the parameters include the number of training batches, number of layers, number of neurons in each layer, learning rate, number of training epochs.

Introduction and Overview

FashionMNIST dataset is a very common data set in machine learning. It includes 10 kinds of different figures which are saved as 28×28 images. Therefore, the shape of its feature is 28×28 . We choose 90% figures' features and labels in the dataset as the training set and the figures' features and labels in the rest of dataset as testing sets randomly. The ultimate goal of the report is to use the FCN (Fully Convolutional Network) method to predict the label of a figure through its feature and the effect will be evaluated by the training loss, validation accuracy and the testing accuracy. Our strategy to improve this effect is to try on different parameters, optimizers, initial conditions and whether the dropout or batch normalization is used.

Theoretical Background

The main method used in the report is the FCN (Fully Convolutional Network) method. Its main propose is to construct hidden layers which includes different number of neurons and then transform the initial data, i.e. the features to the first layer, then the data will be passed through the middle layers and finally reaches the final layer, which is expected to be the labels. Our goal is to find the best way to convert information between neurons in different layers, which will involve a change of dimension, and guarantee a little loss of the information and a good effect on prediction. Such result might be influenced by some basic parameters, for instance, the number of layers and the number of neurons in each layers can affect the loss of the information directly. What's more, since we are not expected to process the large dataset as a whole, our strategy is to only consider parts of the data randomly for the Network and repeat it to improve the effect. During the process, the times we iterate is called the epoch and the rate we adjust the parameter by the loss function is called the learning rate. These are also important factors affecting the effect of the network.

Beyond that, some hyperparameters are alternative and helpful, either. The optimizer, which controls how we learn from the last epoch during the iteration, can be chosen between the RMSProp, Adam and SGD. Here are their principles(θ is parameters in each epoch, and η is the learning rate):

RMSProp Method:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}}$$

Adam Method:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v}_t + \varepsilon}} \widehat{m}_t$$

SGD Method:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

Here, g is the gradient and ε is the smoothing term.^[1]

Moreover, the initial conditions will affect the initial parameters. The dropout strategy can help avoid overfitting by dropping some neurons randomly. The Batch Normalization can help strengthen the stability by normalize the input of each layer.

Algorithm Implementation and Development

In this report, numpy package is used for loading and processing the data, while the matplotlib package is used for visualizing the digit images. Beyond that, the sklearn package provides a convenient way to divide the training set and testing set. What's more, the pytorch package is also important for the report, since it allows us to download the dataset directly and is helpful to construct the network. The pytorch package can allow us to change the parameters and hyperparameters easily either. To help visualizing the rate of progress, the tqdm package is also introduced.

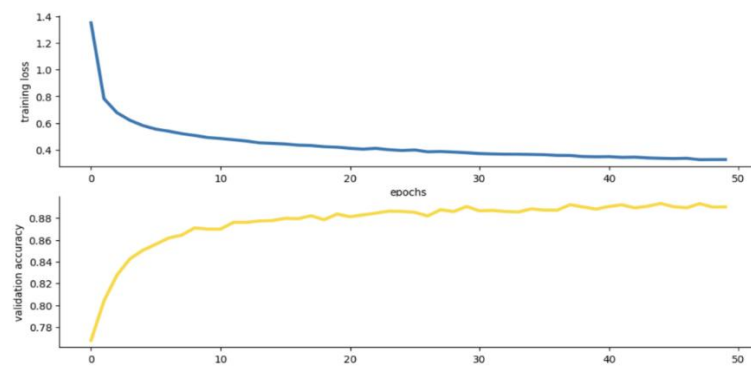
Computational Results

Firstly, we need to download the dataset by pytorch and divide it into the training set and the testing set. Our strategy is to take 90% of data, including their features and labels, randomly as the training set and let the rest of the dataset be the testing set.

Then, we tried to construct the FCN model, it has an adjustable number of hidden layers and neurons in each layer with relu activation, where the first layer is dimension of $28 \times 28 = 784$, the same as the dimension of the feature, and the last

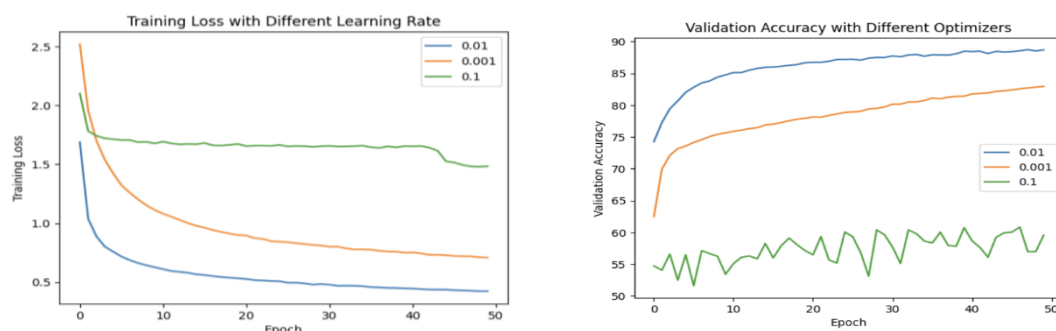
layer is dimension of 10, the same as the dimension of the label. What's more, the number of training batches, learning rate and number of training epochs can all be changed.

After times of trying, we've got an acceptable FCN model which can lead to a good accuracy in a reasonable time. Here are some parameters and hyperparameters we chose. We constructed three hidden layers, each containing 256,218 and 64 neurons. The number of the training batches is 512 and the number of epochs is 50, by the way, for the evaluation, we chose 256 as the number of testing batches. The learning rate is 0.01. What's more, the optimizer we chose is the SGD method and the initial method is Kaiming(HE) Method, which is the default method. Also, we introduced the Dropout with the drop rate 0.5 and the Batch normalization. We used Cross Entropy loss to perform the classification and record the training loss curve as well as include validation of accuracy, on the validation set, at each epoch. Here is the result:



The figure of training loss and validation accuracy at each epoch

The final validation accuracy is 89.04% and the testing accuracy is 88.55%. The running time of the entire program is about 5 minutes, which is a reasonable time. To draw a conclusion, the validation accuracy and the testing accuracy are higher than 85%, which is acceptable, and no overfitting or underfitting exists. Thus, these parameters and hyper parameters will be our baseline parameters for the following steps. Firstly, we want to test which optimizer and learning rate is the most suitable one for our FCN model. Firstly, we used the SGD method and tries different learning rate: 0.01, 0.001 and 0.1. Here is the comparison on the training loss and validation accuracy at each epoch:



The figure of training loss & validation accuracy at each epoch with different learning rate

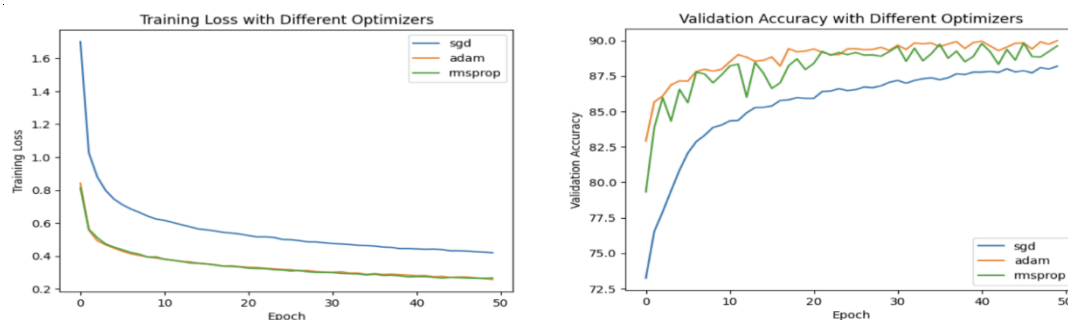
Also, we can compare the final training loss, validation accuracy, test accuracy e of each learning rate:

Learning rate (SVD)	training loss	Validation accuracy	Testing accuracy
0.01	0.4232	88.68%	87.24%
0.001	0.7076	82.95%	81.38%
0.1	1.4846	59.57%	58.77%

final training loss, validation accuracy and test accuracy with different learning rates

It's clear that the SGD optimizer performs best at the learning rate 0.01.

Then, by using the learning rate 0.01, we tried on the RMSProp, Adam and SGD methods:



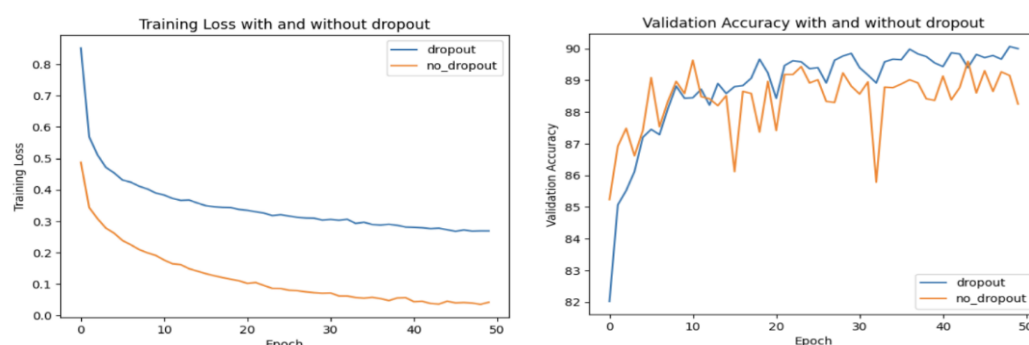
The figure of training loss and validation accuracy at each epoch with different optimizers

optimizer	training loss	Validation accuracy	Testing accuracy	Process time
SGD	0.4232	88.68%	87.24%	293s
Adam	0.2573	89.98%	89.23%	306s
RMSProp	0.2668	89.62%	88.69%	299s

final training loss, validation accuracy, test accuracy and process time with different optimizers

It's obvious that the SGD method has a higher training loss and lower validation accuracies and training accuracies. What's more, the Adam method has the highest accuracy and is much more stable than the RMSProp method between epochs, although it takes a little bit more time to process. Therefore, the Adam method is the most suitable one and we will use the Adam optimizer for the following tests.

After that, we are going to detect how dropout strategy works:



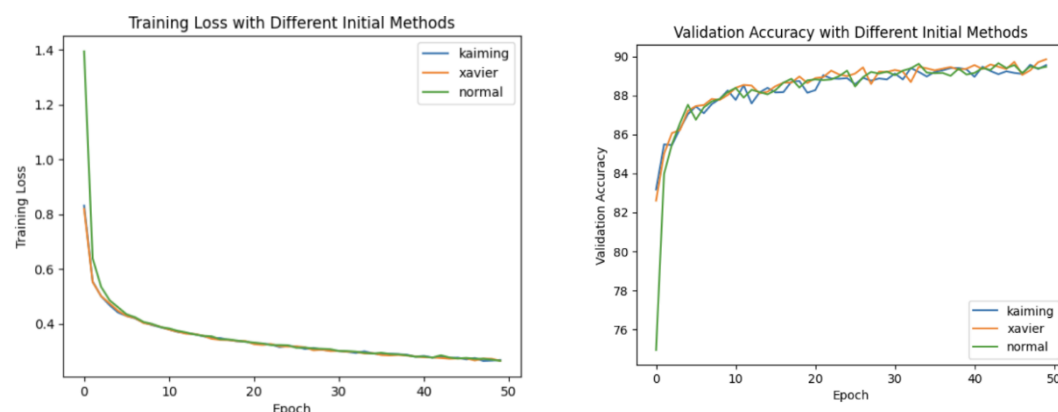
The figure of training loss and validation accuracy at each epoch with and without dropout

	training loss	Validation accuracy	Testing accuracy
With dropout	0.2690	90.00%	88.99%
Without dropout	0.0419	88.25%	87.99%

final training loss, validation accuracy, test accuracy with and without dropout

Here, we used 0.5 as the drop rate. It seems that the dropout helped keep the stability between epochs, although it causes a higher training loss. However, since the Validation accuracy and the testing accuracy are both close with and without dropout, no overfitting or under fitting occurs, the dropout strategy didn't show its ability to avoid overfitting in this model well.

Then, we can test on different initial methods:



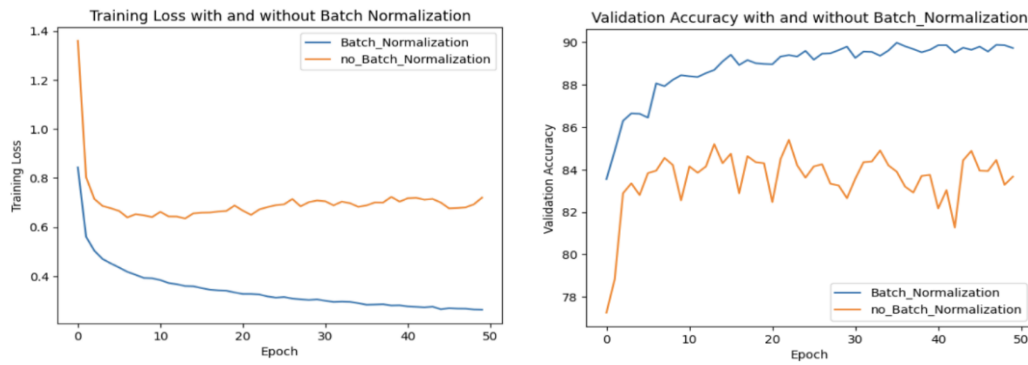
The figure of training loss and validation accuracy at each epoch with different initial methods

Initial method	training loss	Validation accuracy	Testing accuracy
Kaiming Uniform	0.2683	89.55%	88.84%
Xavier Normal	0.2653	89.85%	88.85%
Random Normal	0.2654	89.47%	89.07%

final training loss, validation accuracy, test accuracy and process time with different IC

Although the final testing accuracy, training loss and validation accuracy for the three initial method is similar, it's obvious that if we take random normal as the initial method, the initial training loss is higher and validation accuracy is lower. Above that, it also takes more epochs to converge, which indicates that it's not an ideal method. On contrast, the Kaiming method and the Xavier method performs better and similar.

Last but not the least, we can see how important the Batch normalization is:



The figure of training loss and validation accuracy at each epoch with and without Batch normalization

	training loss	Validation accuracy	Testing accuracy
With Batch normalization	0.2631	89.73%	89.09%
No Batch normalization	0.7204	83.67%	82.65%

final training loss, validation accuracy, test accuracy with and without Batch normalization

It's obvious that the model without Batch normalization has a poor performance. It has a much higher training loss and much lower validation accuracy and testing accuracy. What's more it's also more unstable between epochs, although it might save some time during the training process.

Summary and Conclusions

To draw a conclusion, we succeeded in constructing an FCN model. And after our test, we can take three hidden layers, each containing 256, 218 and 64 neurons and let the number of the training batches be 512 and the number of epochs be 50 as reasonable parameters to ensure a good training accuracy and process speed. What's more, the best optimizer is the Adam optimizer with 0.01 as the learning rate and both Kaiming Uniform and Xavier Normal can be a good initial method. We can also include dropout and Batch normalization to help strengthen the stability of the model and arise the accuracy. By using such model on the FashionMnist dataset, we can have the validation and testing accuracies both higher than 98%.

Acknowledgements

The author is thankful to Prof. Eli Shlizerman for the examples in class, which helped to design the Network programs. What's more, thanks should also be given to the Github Platform, which offered some good program examples.

References

- [1] Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv:1609.04747*.